

UGRAD - Guidelines to the Final Code and Report

The final code should show your work to fine-tune the hyper-parameters (learning rate, epoch, etc.) as well as the model architecture. And in your final notebook, you should have a model having MAE < 2 kcal/mol on the test set. If you've already determined the model in checkpoint 3, all you need to do is to **add some comments** to make the code readable and **clean up** the code as following:

Here are some bugs/issues that some of you may encounter in the previous code:

- **Forgot to re-initialize the model after changing the hyperparameters.**

Wrong code

```
In [ ]: net_H = AtomicNet()
net_C = AtomicNet()
net_N = AtomicNet()
net_O = AtomicNet()

ani_net = torchani.ANIModel([net_H,net_C,net_N,net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)

In [ ]: trainer = ANITrainer(model, 128, 1e-5, 30, 1e-5)
train_loss, val_loss = trainer.train(train_data, val_data, draw_curve=True)

In [ ]: trainer = ANITrainer(model, 128, 1e-3, 30, 1e-5)
train_loss, val_loss = trainer.train(train_data, val_data, draw_curve=True)
```

Correct code

```
In [ ]: net_H = AtomicNet()
net_C = AtomicNet()
net_N = AtomicNet()
net_O = AtomicNet()

ani_net = torchani.ANIModel([net_H,net_C,net_N,net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)

In [ ]: trainer = ANITrainer(model, 128, 1e-5, 30, 1e-5)
train_loss, val_loss = trainer.train(train_data, val_data, draw_curve=True)

In [ ]: net_H = AtomicNet()
net_C = AtomicNet()
net_N = AtomicNet()
net_O = AtomicNet()

ani_net = torchani.ANIModel([net_H,net_C,net_N,net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)

In [ ]: trainer = ANITrainer(model, 128, 1e-3, 30, 1e-5)
train_loss, val_loss = trainer.train(train_data, val_data, draw_curve=True)
```

In the wrong code, when you change the hyper-parameter without re-initialize the model, the trainer will continuously train the model from the previous trial, which is already trained for some epochs. This will make the comparison unfair.

- **DON'T plot the validation dataset every epoch.** To do this, set the `draw_plot` to `False` in the training code.

```
class ANITrainer:
    def __init__(self, model, batch_size, learning_rate, epoch, l2):
        self.model = model

        num_params = sum(item.numel() for item in model.parameters())
        print(f"{model.__class__.__name__} - Number of parameters: {num_params}")

        self.batch_size = batch_size
        self.optimizer = torch.optim.Adam(model.parameters(), learning_rate, weight_decay=l2)
        self.epoch = epoch

    def train(self, train_data, val_data, early_stop=True, draw_curve=True):
        self.model.train()

        # init data loader
        print("Initialize training data...")
        train_data_loader = train_data.collate(self.batch_size).cache()

        # definition of loss function: MSE is a good choice!
        loss_func = nn.MSELoss()

        # record epoch losses
        train_loss_list = []
        val_loss_list = []
        lowest_val_loss = np.inf

        for i in tqdm(range(self.epoch), leave=True):
            train_epoch_loss = 0.0
            for train_data_batch in train_data_loader:
                ...

            # use the self.evaluate to get loss on the validation set
            val_epoch_loss = self.evaluate(val_data, draw_plot=False)

        # append the losses
```

- **DON'T output the list of train/validation loss for every epoch after training.** To do this, use some variables name to take the return values of `trainer.train()` function.

Wrong code:

```
In [ ]: trainer.train(...)
```

Correct code:

```
In [ ]: train_losses, val_losses = trainer.train(...)
```

For the final report, you can refer to the examples on bCourses, but generally a good report should contain the following part:

1. **Introduction and overview of the method:** what scientific problems we are trying to solve, what dataset do we use, what architecture do we use, etc.
2. **Results:**

- A table that shows all combinations of the hyper-parameters you tried and corresponding results. A sample looks like:

Hidden layers	Learning rate	Epochs	L2 Regularization	Dropouts	Train MAE (kcal/mol)	Validation MAE (kcal/mol)	Test MAE (kcal/mol)
[384, 128, 1]	1e-3	30	1e-5	0.0	2.23	2.83	3.92
[384, 128, 1]	1e-4	30	1e-4	0.1	1.83	3.32	1.10

- Learning curve of the best model, i.e. training/validation loss (or some other metrics) vs. training epochs
- If you want, you can also report a detailed analysis upon the dataset, such as in the example, the author reported errors on sub-dataset containing 1,2,3,4 heavy atoms respectively. *Hint: if you want to do this, you can take usage of the “species” variable.*

3. **Conclusions:** describe your findings in the project.

4. **References**